



**KAZAKH-BRITISH
TECHNICAL
UNIVERSITY**

Course name: Databases

Laboratory work 4

Submission format: please work with the database from the prev lab work and provide fullpage screenshots that covers the code and results of each task in the PDF format, also provide sql files.

Required tasks:

1. Retrieve all airline names in uppercase and display them together with their country

The screenshot shows the DataGrip interface with a SQL query executed in the console. The query is:

```
-- 1.Retrieve all airline names in uppercase and display them together with their countries.
select 1s 527 ms
      upper(airline_name) as uppercase_airline_name, airline_country
from airline;
```

The results are displayed in a table with two columns: `uppercase_airline_name` and `airline_country`. The table contains 10 rows of data:

uppercase_airline_name	airline_country
AIREASY	France
FLYHIGH	Brazil
FLYFLY	Poland
KAZAIR	Turkey
AIR ASTANA	Kazakhstan
SCAT AIRLINES	Kazakhstan
TURKISH AIRLINES	Turkey
PEGASUS AIRLINES	Turkey
LUFTHANSA	Germany
...	...

2. Replace any occurrence of the word "Air" in airline names with "Aero" and display both the original and modified airline names.

The screenshot shows the DataGrip interface with a SQL console. The Database Explorer on the left shows a database named 'university_db@localhost' with tables: 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', and 'seat'. The console contains a SQL query to replace 'Air' with 'Aero' in airline names. The output shows a table with two columns: 'original_name' and 'modified_name'.

```

/* 2. Replace any occurrence of the word "Air" in airline names with "Aero" and display both the
original and modified airline names. */

select
  airline_name as original_name,
  replace(airline_name, 'air', 'aero') as modified_name
from airline;

```

original_name	modified_name
1 AirEasy	AirEasy
2 FlyHigh	FlyHigh
3 FlyFly	FlyFly
4 KazAir	KazAir
5 Air Astana	Air Astana
6 SCAT Airlines	SCAT Airlines
7 Turkish Airlines	Turkish Airlines
8 Pegasus Airlines	Pegasus Airlines
9 Lufthansa	Lufthansa

3. Find all flight numbers that coordinates with both airline 1 and airline 2.

The screenshot shows the DataGrip interface with a SQL console. The Database Explorer on the left shows a database named 'university_db@localhost' with tables: 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', and 'seat'. The console contains a SQL query to find flight numbers for airline 1 and airline 2. The output shows a table with four columns: 'airline1_flight', 'airline2_flight', 'departing_airport_id', and 'arriving_airport_id'.

```

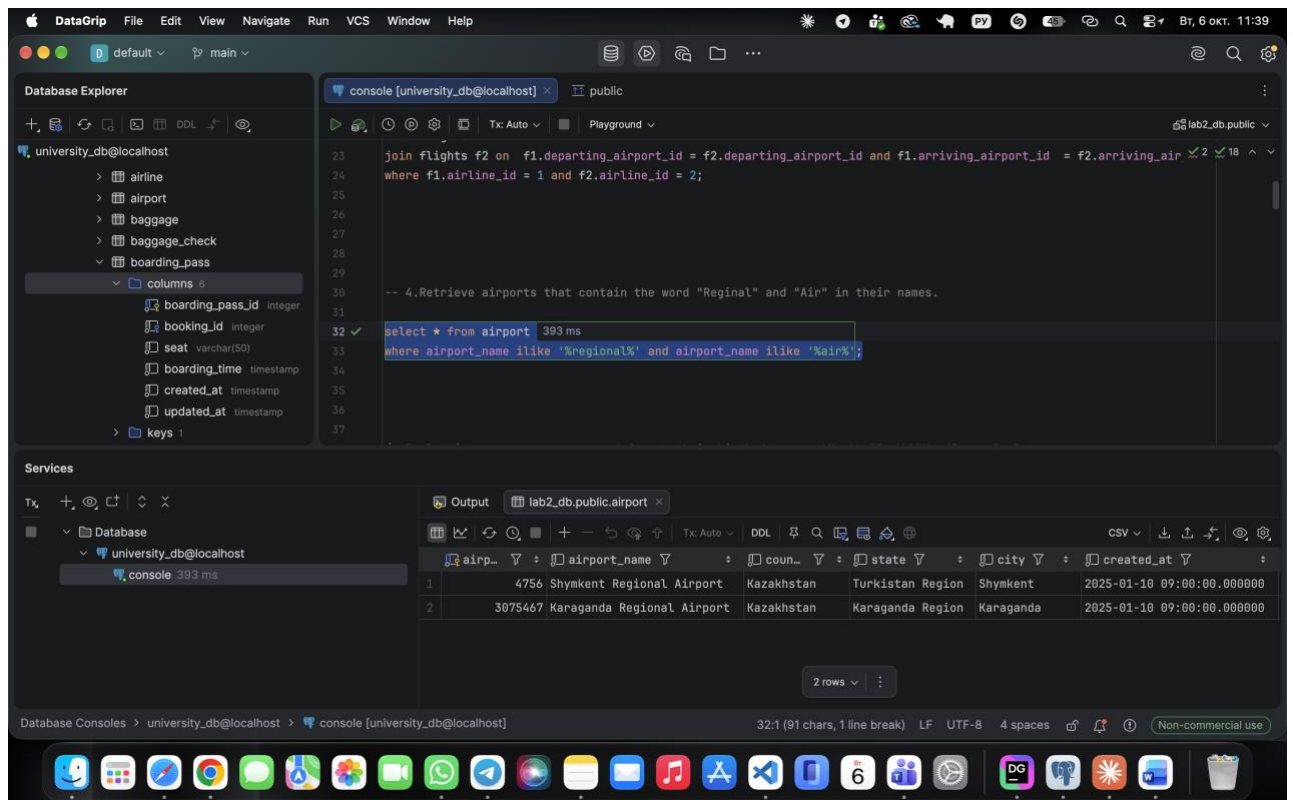
-- 3. Find all flight numbers that coordinates with both airline 1 and airline 2.

select f1.flight_id as airline1_flight, f2.flight_id as airline2_flight, f1.departing_airport_id, f1.arriving_airport_id
from flights f1
join flights f2 on f1.departing_airport_id = f2.departing_airport_id and f1.arriving_airport_id = f2.arriving_airport_id
where f1.airline_id = 1 and f2.airline_id = 2;

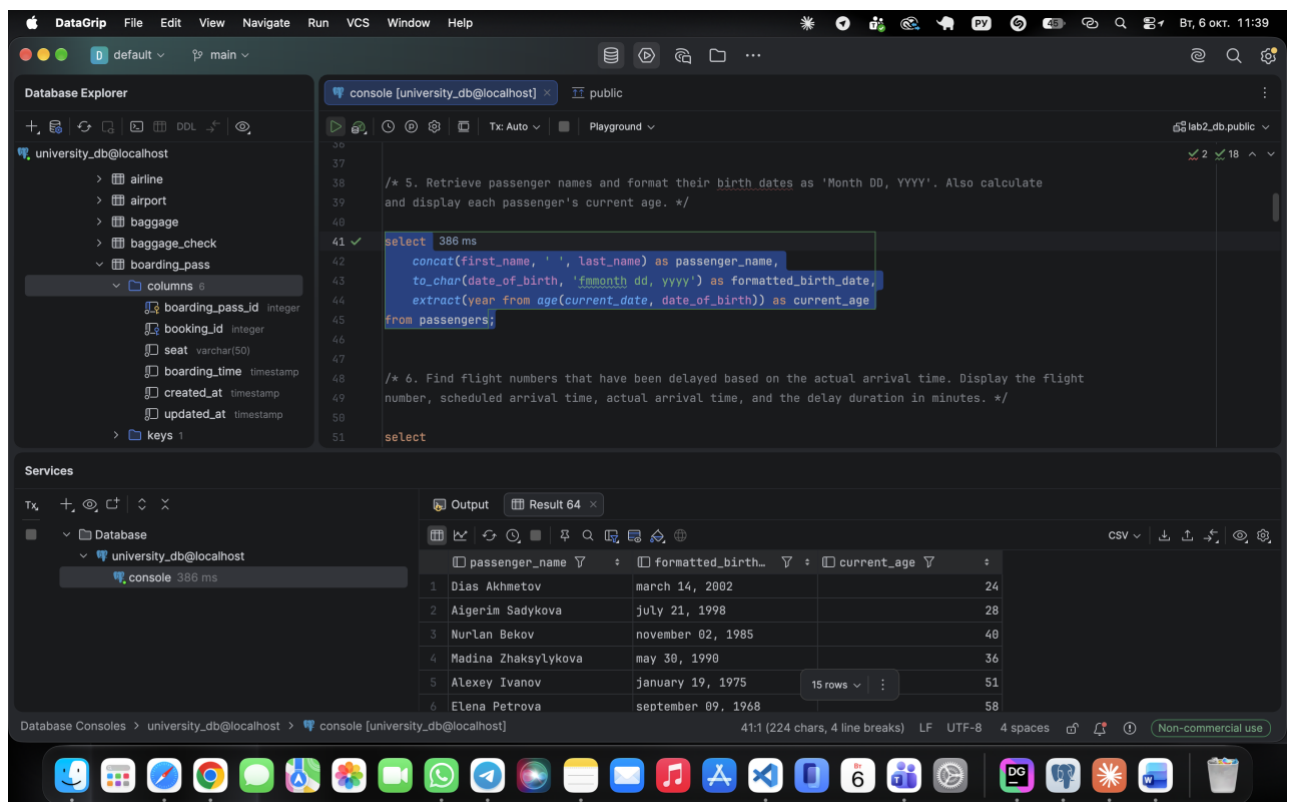
```

airline1_flight	airline2_flight	departing_airport_id	arriving_airport_id
1 6	41	1	
2 12	41	1	
3 18	41	1	
4 24	41	1	
5 30	41	1	
6 36	41	1	

4. Retrieve airports that contain the word "Reginal" and "Air" in their names.



- Retrieve passenger names and format their birth dates as 'Month DD, YYYY'. Also calculate and display each passenger's current age.



6. Find flight numbers that have been delayed based on the actual arrival time. Display the flight number, scheduled arrival time, actual arrival time, and the delay duration in minutes.

The screenshot shows the DataGrip IDE interface. On the left, the 'Database Explorer' pane shows the 'university_db@localhost' database with a tree structure including 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', and 'columns'. The 'columns' table is selected, showing its schema: 'boarding_pass_id' (integer), 'booking_id' (integer), 'seat' (varchar(50)), 'boarding_time' (timestamp), 'created_at' (timestamp), and 'updated_at' (timestamp). The 'console [university_db@localhost]' pane shows a SQL query for finding delayed flights. The query is as follows:

```
44      extract(year from age(current_date, date_of_birth)) as current_age
45  from passengers;
46
47
48  /* 6. Find flight numbers that have been delayed based on the actual arrival time. Display the flight
49  number, scheduled arrival time, actual arrival time, and the delay duration in minutes. */
50
51  select 440 ms
52      flight_id, sch_arrival_time as scheduled_arrival, act_arrival_time as actual_arrival,
53      extract(epoch from (act_arrival_time - sch_arrival_time)) / 60 as delay_minutes
54  from flights
55  where act_arrival_time > sch_arrival_time;
56
57
58
59  /* 7. Create a query that divides passengers into age groups like 'Young' and 'Adult' based on their
```

The 'Output' pane shows the results of the query, which is a table with 4 columns: 'flight_id', 'scheduled_arrival', 'actual_arrival', and 'delay_minutes'. The table contains 7 rows of data:

flight_id	scheduled_arrival	actual_arrival	delay_minutes
1	2026-01-01 18:00:00.000000	2026-01-01 18:05:00.000000	
2	2026-01-02 04:00:00.000000	2026-01-02 04:13:00.000000	
3	2026-01-02 14:00:00.000000	2026-01-02 14:21:00.000000	
4	2026-01-02 20:00:00.000000	2026-01-02 20:29:00.000000	
5	2026-01-03 06:00:00.000000	2026-01-03 06:32:00.000000	
6	2026-01-04 02:00:00.000000	2026-01-04 02:06:00.000000	
7	2026-01-04 02:00:00.000000	2026-01-04 02:06:00.000000	

7. Create a query that divides passengers into age groups like 'Young' and 'Adult' based on their birth date. Young passengers age between 18 and 35, Adult passengers age between 36 and 55. Also count how many people we have in each group.

The screenshot shows the DataGrip interface with a SQL console. The left pane displays the database schema for 'university_db@localhost', including tables like 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', and 'passengers'. The main console shows a SQL query (Task 7) that categorizes passengers into 'Young' and 'Adult' age groups based on their birth date. The query uses a CTE named 'passenger_groups' and a CASE statement. The results pane shows two rows: 'adult' with 6 passengers and 'young' with 7 passengers.

```

/* 7. Create a query that divides passengers into age groups like 'Young' and 'Adult' based on their
birth date. Young passengers age between 18 and 35, Adult passengers age between 36 and 55.
Also count how many people we have in each group. */

with passenger_groups as (
  select
    passenger_id,
    case
      when extract(year from age(current_date, date_of_birth)) between 18 and 35 then 'young'
      when extract(year from age(current_date, date_of_birth)) between 36 and 55 then 'adult'
      else 'other'
    end as age_group
  from passengers
)
select
  age_group,
  count(*) as total_passengers
from passenger_groups
group by age_group

```

age_group	total_passengers
adult	6
young	7

8. Create a query that categorizes ticket prices based on their price as "Cheap," "Medium" or "Expensive."

The screenshot shows the DataGrip interface with a SQL console. The left pane displays the database schema for 'university_db@localhost'. The main console shows a SQL query (Task 8) that categorizes ticket prices into 'cheap', 'medium', or 'expensive' based on their value. The query uses a CASE statement. The results pane shows six rows of booking data with their respective ticket prices and categories.

```

/* 8. Create a query that categorizes ticket prices based on their price as "Cheap," "Medium" or
"Expensive." */

select
  booking_id, ticket_price,
  case
    when ticket_price < 100 then 'cheap'
    when ticket_price between 100 and 300 then 'medium'
    else 'expensive'
  end as price_category
from booking;

```

booking_id	ticket_price	price_category
1	77.01	cheap
2	114.02	medium
3	151.03	medium
4	188.04	medium
5	262.06	medium
6	262.06	medium

9. Find number of airline names in each airline country and display only countries that have more than one airline.

The screenshot shows the DataGrip IDE interface. On the left, the 'Database Explorer' pane shows the 'university_db@localhost' database with a tree structure including 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', and 'columns'. The 'columns' table is selected, showing columns like 'boarding_pass_id', 'booking_id', 'seat', 'boarding_time', 'created_at', and 'updated_at'. The main editor shows a SQL query in the 'console [university_db@localhost]' pane. The query is as follows:

```
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102
```

The query is a SELECT statement that counts the number of airline names in each country, filtered by a price category. The results are shown in the 'Output' pane, which displays a table with two columns: 'airline_country' and 'total_airlines'. The results are as follows:

airline_country	total_airlines
Turkey	3
Kazakhstan	2

10. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time. Also calculate the percentage of delayed flights for each airline.

The screenshot shows the DataGrip interface with a SQL query in the console. The query calculates the percentage of delayed flights for each airline. The results table shows 6 rows of data.

```

104  /* 10. Find flights that arrived late according to their actual arrival time compared to the scheduled
105  arrival time. Also calculate the percentage of delayed flights for each airline. */
106  select 412 ms
107  a.airline_id, a.airline_name,
108  count(case when f.act_arrival_time > f.sch_arrival_time then 1 end) as delayed_flights,
109  count(f.flight_id) as total_flights,
110  round(
111    (count(case when f.act_arrival_time > f.sch_arrival_time then 1 end)::decimal / nullif(count(f.flight_id), 0)) * 100,
112    2
113  ) as delayed_percentage
114  from airline a
115  left join flights f 1<->0..n: on a.airline_id = f.airline_id
116  group by a.airline_id, a.airline_name;
117
118

```

airline_id	airline_name	delayed_flights	total_flights	delayed_percentage
1	1002 AirEasy	0	0	0
2	1003 FlyHigh	0	0	0
3	1004 FlyFly	0	0	0
4	3 Turkish Airlines	7	7	100
5	1001 KazAir	0	0	0
6	5 Lufthansa	7	7	100

11. Find each airline's total number of flights and average flight delay. Display only airlines that have at least 3 flights and an average delay greater than 10 minutes.

The screenshot shows the DataGrip interface with a SQL query for finding airlines with at least 3 flights and an average delay greater than 10 minutes. The results table shows 4 rows of data.

```

119  /* 11. Find each airline's total number of flights and average flight delay. Display only airlines that
120  have at least 3 flights and an average delay greater than 10 minutes. */
121
122  select 433 ms
123  a.airline_id, a.airline_name,
124  count(f.flight_id) as total_flights,
125  avg(
126    case
127      when f.act_arrival_time > f.sch_arrival_time
128      then extract(epoch from (f.act_arrival_time - f.sch_arrival_time)) / 60
129      else 0
130    end
131  ) as avg_delay_minutes
132  from airline a
133  join flights f 1<->1..n: on a.airline_id = f.airline_id

```

airline_id	airline_name	total_flights	avg_delay_minutes
1	3 Turkish Airlines	7	13.1428571428571429
2	5 Lufthansa	7	27
3	4 Pegasus Airlines	7	20.4285714285714286
4	6 Emirates	6	33.6666666666666667

12. Find the top 3 most expensive tickets and display the passenger name, flight number, airline name, and ticket price.

The screenshot shows the DataGrip application interface. On the left, the 'Database Explorer' pane displays the schema of the 'university_db@localhost' database, including tables like 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', 'columns', 'boarding_pass_id', 'booking_id', 'seat', 'boarding_time', 'created_at', 'updated_at', and 'keys'. The main editor displays a SQL query for finding the top 3 most expensive tickets. The query is as follows:

```
/* 12. Find the top 3 most expensive tickets and display the passenger name, flight number, airline name, and ticket price. */  
  
select  
    concat(p.first_name, ' ', p.last_name) as passenger_name,  
    f.flight_id, a.airline_name, b.ticket_price  
from booking b  
join passengers p 1.n<->1: on b.passenger_id = p.passenger_id  
join flights f on b.flight_id = f.flight_id  
join airline a 1.n<->1: on f.airline_id = a.airline_id  
order by b.ticket_price desc  
limit 3;
```

The 'Output' pane at the bottom right shows the results of the query, displaying 3 rows of data in a table format:

	passenger_name	flight_id	airline_name	ticket_price
1	Saltanat Omarova	38	Turkish Airlines	487.37
2	Yerlan Abenov	13	SCAT Airlines	484.12
3	Alexey Ivanov	10	Lufthansa	478.49

13. Calculate the total amount spent by each passenger on flight tickets. Display the passenger's name, number of purchased tickets, total amount spent, and average ticket price.

The screenshot shows the DataGrip IDE interface. On the left, the 'Database Explorer' pane shows a database named 'university_db@localhost' with tables like 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', and 'columns'. The 'columns' table is selected, showing columns like 'boarding_pass_id', 'booking_id', 'seat', 'boarding_time', 'created_at', and 'updated_at'. The 'console [university_db@localhost]' pane shows a SQL query that concatenates passenger names, counts tickets, sums ticket prices, and calculates the average ticket price. The 'Output' pane shows the results of the query, displaying a table with columns: 'passenger_name', 'tickets_purchased', 'total_amount_spent', and 'average_ticket_price'. The results are sorted by 'total_amount_spent' in descending order, showing the top 6 passengers.

```

160 name, number of purchased tickets, total amount spent, and average ticket price. */
161
162 select 395 ms
163   concat(p.first_name, ' ', p.last_name) as passenger_name,
164   count(b.booking_id) as tickets_purchased,
165   sum(b.ticket_price) as total_amount_spent,
166   round(avg(b.ticket_price), 2) as average_ticket_price
167 from passengers p
168 join booking b 1<->1.n on p.passenger_id = b.passenger_id
169 group by p.passenger_id, p.first_name, p.last_name;
170
171
172 /* 14. Analyze airport traffic. For each airport, calculate the number of departing flights, the number
173 of arriving flights, and the total number of flights. Display only airports with more than 10 total
174 flights and sort them by total traffic in descending order. */

```

	passenger_name	tickets_purchased	total_amount_spent	average_ticket_price
1	Madina Zhaksylykova	4	1169.02	292.25
2	Dana Nurpeisova	4	1135.26	283.81
3	Elena Petrova	4	1004.1	251.02
4	Kamila Sultanova	4	805.42	201.35
5	Yerlan Abenov	4	1118.38	279.59
6	Aiqerim Sadykova	4	872.94	218.23

14. Analyze airport traffic. For each airport, calculate the number of departing flights, the number of arriving flights, and the total number of flights. Display only airports with more than 10 total flights and sort them by total traffic in descending order.

DataGrip File Edit View Navigate Run VCS Window Help

console [university_db@localhost] x public

Database Explorer

university_db@localhost

- airline
- airport
- baggage
- baggage_check
- boarding_pass
 - columns 6
 - boarding_pass_id integer
 - booking_id integer
 - seat varchar(50)
 - boarding_time timestamp
 - created_at timestamp
 - updated_at timestamp
 - keys 1

```
select
  ap.airport_id, ap.airport_name,
  count(case when f_dep.departing_airport_id is not null then 1 end) as departing_flights,
  count(case when f_arr.arriving_airport_id is not null then 1 end) as arriving_flights,
  (count(case when f_dep.departing_airport_id is not null then 1 end) +
   count(case when f_arr.arriving_airport_id is not null then 1 end)) as total_flights
from airport ap
left join flights f_dep 1<->0.n on ap.airport_id = f_dep.departing_airport_id
left join flights f_arr on ap.airport_id = f_arr.arriving_airport_id
group by ap.airport_id, ap.airport_name
having (count(case when f_dep.departing_airport_id is not null then 1 end) +
        count(case when f_arr.arriving_airport_id is not null then 1 end)) > 10
order by total_flights desc;
```

Output Result 73

airport_id	airport_name	departing_flights	arriving_flights	total_flights
1	4 Airport_4	56	56	112
2	3 Airport_3	49	49	98
3	5 Airport_5	49	49	98
4	1 Airport_1	49	49	98
5	2 Airport_2	48	48	96
6	6 Airport_6	42	42	84

Database Consoles > university_db@localhost > console [university_db@localhost]

176:1 (770 chars, 12 line breaks) LF UTF-8 4 spaces Non-commercial use